

# Guía: excepciones predefinidas de Oracle

Sesión 1.3.1 · Indicador de logro 1.3 · Modelo de datos CLÍNICA KETEKURA

|                    |                                       |                      |                  |
|--------------------|---------------------------------------|----------------------|------------------|
| <b>Asignatura</b>  | BDY1103 – Taller de Base de Datos     | <b>Experiencia</b>   | EA1              |
| <b>Tiempo</b>      | 1 h 20 min (clase) + trabajo autónomo | <b>Modalidad</b>     | Individual       |
| <b>Herramienta</b> | Oracle SQL Developer                  | <b>Base de datos</b> | CLÍNICA KETEKURA |

## 1. El bloque que funciona hasta que no

Este bloque calcula el descuento de tercera edad que le corresponde a un paciente de la clínica. Está bien escrito y funciona.

```

DECLARE
    v_edad      NUMBER(3);
    v_descuento PORC_DESCTO_3RA_EDAD.porcentaje_desccto%TYPE;
BEGIN
    SELECT TRUNC(MONTHS_BETWEEN(SYSDATE, fecha_nacimiento)/12)
    INTO   v_edad
    FROM   PACIENTE
    WHERE  pac_run = 7599430;          -- paciente de 65 años

    SELECT porcentaje_desccto INTO v_descuento
    FROM   PORC_DESCTO_3RA_EDAD
    WHERE  v_edad BETWEEN anno_ini AND anno_ter;

    DBMS_OUTPUT.PUT_LINE('Descuento: ' || v_descuento || '%');
END;
/

```

Cambia un solo dato, el RUN, por el del paciente 6215470, y vuelve a ejecutarlo.

ORA-01403: no data found

El bloque no está mal escrito. Ese paciente tiene 4 años y no cae en ningún tramo de la tabla de descuentos, que parte en 65. La consulta es correcta, la lógica es correcta, y aun así el programa se cae.

**Un programa que solo contempla el caso feliz no es un programa terminado. La sección EXCEPTION es lo que convierte un bloque correcto en un bloque confiable.**

## Qué es una excepción

Una excepción es una condición de error que ocurre durante la ejecución del bloque, no durante su compilación. Cuando se produce, PL/SQL interrumpe de inmediato el cuerpo del programa y busca a alguien que se haga cargo.

|                               | Error de compilación             | Excepción                                                              |
|-------------------------------|----------------------------------|------------------------------------------------------------------------|
| Cuándo aparece                | Al compilar el bloque            | Al ejecutarlo                                                          |
| Depende de los datos          | No                               | Muchas veces sí: el mismo bloque funciona con un dato y falla con otro |
| Ejemplo                       | Escribir SELECT en vez de SELECT | Buscar un paciente que no existe                                       |
| Prefijo típico                | PLS-                             | ORA-                                                                   |
| Se puede manejar en el código | No: hay que corregirlo           | Sí: para eso existe la sección EXCEPTION                               |

## 2. La sección EXCEPTION

Un bloque PL/SQL tiene tres secciones, y la tercera es opcional. Hasta ahora solo has usado las dos primeras.

```
DECLARE
  -- declaraciones
BEGIN
  -- cuerpo: lo que el programa hace cuando todo sale bien
EXCEPTION
  WHEN nombre_de_la_excepcion THEN
    -- que hacer si ocurre ESA condicion
  WHEN otra_excepcion THEN
    -- que hacer si ocurre esta otra
  WHEN OTHERS THEN
    -- red de seguridad: siempre al final
END;
```

### Cómo fluye el control

Este punto es el que más cuesta, y conviene entenderlo antes que la sintaxis.

1. El bloque ejecuta su cuerpo normalmente, línea por línea.
2. Si una línea provoca una excepción, PL/SQL abandona el cuerpo en ese punto.
3. Busca en la sección EXCEPTION un WHEN que corresponda a esa excepción.
4. Si lo encuentra, ejecuta ese manejador y después el bloque termina normalmente.
5. Si no lo encuentra, la excepción se propaga al bloque que lo contiene, o al usuario si no hay ninguno.

**El malentendido más frecuente:** la sección EXCEPTION no es un IF. Cuando se ejecuta un manejador, el control NO vuelve a la línea donde ocurrió el error, ni a la siguiente. Las instrucciones que quedaron después del punto de falla no se ejecutan nunca.

Compruébalo tú mismo. Ejecuta este bloque con el paciente 6215470 y observa cuál de los dos mensajes aparece:

```
BEGIN
  SELECT porcentaje_desccto INTO v_descuento
  FROM   PORC_DESCCTO_3RA_EDAD
  WHERE  4 BETWEEN anno_ini AND anno_ter;

  DBMS_OUTPUT.PUT_LINE('Esta linea nunca se ejecuta.');
```

EXCEPTION

```
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Esta si.');
```

END;

/

### 3. Las excepciones predefinidas que vas a usar

Oracle tiene un conjunto de excepciones con nombre propio. No hay que declararlas: se usan directamente en el WHEN.

| Nombre              | Número    | Se produce cuando...                                              |
|---------------------|-----------|-------------------------------------------------------------------|
| NO_DATA_FOUND       | ORA-01403 | Un SELECT ... INTO no devuelve ninguna fila                       |
| TOO_MANY_ROWS       | ORA-01422 | Un SELECT ... INTO devuelve más de una fila                       |
| ZERO_DIVIDE         | ORA-01476 | Se intenta dividir por cero                                       |
| DUP_VAL_ON_INDEX    | ORA-00001 | Un INSERT o UPDATE repite un valor de clave primaria o única      |
| VALUE_ERROR         | ORA-06502 | Un dato no cabe en la variable, o falla una conversión en PL/SQL  |
| INVALID_NUMBER      | ORA-01722 | Falla la conversión de texto a número dentro de una sentencia SQL |
| INVALID_CURSOR      | ORA-01001 | Se opera sobre un cursor que no está abierto                      |
| CURSOR_ALREADY_OPEN | ORA-06511 | Se abre un cursor que ya estaba abierto                           |
| CASE_NOT_FOUND      | ORA-06592 | Ningún WHEN de un CASE se cumplió y no hay ELSE                   |
| OTHERS              | —         | Cualquier excepción que no tenga un WHEN propio. Siempre al final |

#### Ejemplos sobre el modelo de la clínica

##### NO\_DATA\_FOUND

```
-- El paciente 6215470 tiene 4 años: no cae en ningún tramo.  
SELECT porcentaje_desccto INTO v_descuento  
FROM   PORC_DESCCTO_3RA_EDAD  
WHERE  v_edad BETWEEN anno_ini AND anno_ter;
```

##### TOO\_MANY\_ROWS

```
-- El médico 5588583 tiene DOS especialidades registradas.  
SELECT esp_id INTO v_esp  
FROM   ESPECIALIDAD_MEDICO  
WHERE  med_run = 5588583;
```

Fíjate en la asimetría: un SELECT ... INTO falla tanto si trae cero filas como si trae dos. Solo acepta exactamente una. Cuando pueden venir varias, la herramienta correcta es un cursor, no un manejador de excepciones.

##### ZERO\_DIVIDE

```
-- 21 de los 51 médicos no tienen ninguna atención registrada.  
v_promedio := v_total_costo / v_cantidad_atenciones;
```

##### DUP\_VAL\_ON\_INDEX

```
-- PAGO_MOROSO tiene clave primaria (pac_run, ate_id).  
-- La segunda vez que insertes el mismo par, falla.  
INSERT INTO PAGO_MOROSO VALUES (...);
```

## 4. WHEN OTHERS, SQLCODE y SQLERRM

OTHERS captura cualquier excepción que no tenga un manejador propio. Es la red de seguridad, y tiene dos reglas.

- **Va siempre al último.** Si lo pones antes de otro WHEN, Oracle rechaza el bloque con PLS-00370, porque los manejadores posteriores serían inalcanzables.
- **Nunca debe quedar mudo.** Como OTHERS no sabe qué ocurrió, hay que preguntárselo a Oracle con dos funciones.

| Función | Devuelve                                                                |
|---------|-------------------------------------------------------------------------|
| SQLCODE | El número del error. Es negativo, salvo NO_DATA_FOUND, que devuelve 100 |
| SQLERRM | El texto del mensaje de error asociado                                  |

```
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('El paciente no tiene descuento por edad.');
```

  

```
  WHEN TOO_MANY_ROWS THEN
    DBMS_OUTPUT.PUT_LINE('Hay mas de un tramo que calza. Revisar la tabla.');
```

  

```
  WHEN OTHERS THEN                                -- siempre al final
    DBMS_OUTPUT.PUT_LINE('Error inesperado.');
```

```
    DBMS_OUTPUT.PUT_LINE('Codigo : ' || SQLCODE);
    DBMS_OUTPUT.PUT_LINE('Mensaje: ' || SQLERRM);
END;
```

**La peor línea del PL/SQL:** WHEN OTHERS THEN NULL; El bloque deja de fallar, sí, pero también deja de avisar. Un proceso puede correr meses sin procesar nada y nadie se entera. Si capturas OTHERS, informa siempre qué pasó.

### Un detalle sobre SQLCODE

Casi todos los códigos son negativos: ORA-01422 devuelve -1422. La excepción es NO\_DATA\_FOUND, que devuelve 100 en vez de -1403. Es una particularidad histórica de Oracle y conviene conocerla, porque rompe cualquier comprobación del tipo «si SQLCODE es negativo».

## 5. Dónde poner el manejador: el bloque anidado

Imagina que recorres los 51 médicos calculando el costo promedio de sus atenciones. Veintiuno no tienen ninguna, así que la división falla.

### Manejador en el bloque exterior: el proceso se detiene

```
BEGIN
  FOR v_med IN c_medicos LOOP
    v_promedio := v_total / v_cantidad;      -- falla en el primer medico sin
    atenciones
    DBMS_OUTPUT.PUT_LINE(...);
  END LOOP;
EXCEPTION
  WHEN ZERO_DIVIDE THEN
    DBMS_OUTPUT.PUT_LINE('Hubo una division por cero.');
```

El ciclo se abandona por completo. Los médicos que venían después no se procesan nunca, y el informe queda a medias.

### Manejador en un bloque anidado: el proceso continúa

```
BEGIN
  FOR v_med IN c_medicos LOOP

    BEGIN                                -- bloque anidado, por cada vuelta
      v_promedio := v_total / v_cantidad;
      DBMS_OUTPUT.PUT_LINE('Promedio: ' || v_promedio);
    EXCEPTION
      WHEN ZERO_DIVIDE THEN
        DBMS_OUTPUT.PUT_LINE('Sin atenciones registradas.');
```

**Esta es la diferencia entre un informe incompleto y un informe completo.** El bloque anidado acota el daño a una sola vuelta del ciclo: la excepción se maneja ahí y el FOR sigue con el médico siguiente.

**Un bloque anidado puede ir en cualquier parte:** dentro del cuerpo de otro bloque, dentro de un ciclo, e incluso dentro de una sección EXCEPTION. No necesita DECLARE si no declara nada: basta con BEGIN ... EXCEPTION ... END;

## Propagación

Si un bloque anidado no maneja la excepción, esta sube al bloque que lo contiene, y así sucesivamente hasta que alguien la capture o hasta que llegue al usuario. Esa cadena se llama propagación, y es la razón por la que puedes decidir dónde manejar cada error: cerca, para continuar; lejos, para abortar.

## 6. Errores frecuentes

| Error                                          | Síntoma                                               | Cómo se corrige                                                                |
|------------------------------------------------|-------------------------------------------------------|--------------------------------------------------------------------------------|
| WHEN OTHERS antes de otro WHEN                 | PLS-00370: OTHERS debe ser el último manejador        | Mover OTHERS al final de la sección                                            |
| WHEN OTHERS THEN NULL                          | El bloque no falla, pero tampoco informa nada         | Agregar SQLCODE y SQLERRM como mínimo                                          |
| Escribir el número en vez del nombre           | PLS-00201: identificador ORA-01403 no declarado       | En el WHEN va el nombre; el número lo entrega SQLCODE                          |
| Esperar que el bloque continúe donde falló     | Faltan líneas de salida sin explicación               | El control no vuelve. Si hay que continuar, usar un bloque anidado             |
| Manejar en el exterior dentro de un ciclo      | El proceso se detiene en la primera fila con problema | Mover el manejador a un bloque anidado dentro del ciclo                        |
| Esperar NO_DATA_FOUND de un cursor             | La excepción nunca llega                              | Un cursor sin filas no lanza nada: el ciclo no entra. Llevar un contador       |
| Usar EXCEPTION para validar reglas del negocio | No existe la excepción predefinida que se necesita    | Eso corresponde a excepciones definidas por el usuario, en la sesión siguiente |

## 7. Resumen para consultar rápido

| Necesito...                            | Escribo...                                    |
|----------------------------------------|-----------------------------------------------|
| Abrir la sección de manejo             | EXCEPTION (después del cuerpo, antes del END) |
| Capturar una excepción concreta        | WHEN NO_DATA_FOUND THEN ...                   |
| Capturar varias en el mismo manejador  | WHEN NO_DATA_FOUND OR TOO_MANY_ROWS THEN ...  |
| Capturar todo lo demás                 | WHEN OTHERS THEN ... (siempre al final)       |
| Saber qué número de error ocurrió      | SQLCODE                                       |
| Saber qué mensaje corresponde          | SQLERRM                                       |
| Acotar el error a una vuelta del ciclo | BEGIN ... EXCEPTION ... END; dentro del LOOP  |
| Dejar que el error suba                | No escribir un WHEN para él                   |

## 8. Las tablas de KETEKURA que usarás

| Tabla                | Columnas relevantes para esta guía                                                                                       |
|----------------------|--------------------------------------------------------------------------------------------------------------------------|
| PACIENTE             | pac_run, dv_run, pnombre, snombre, apaterno, amaterno, fecha_nacimiento, telefono, sal_id                                |
| MEDICO               | med_run, dv_run, pnombre, apaterno, amaterno, sueldo_base, fecha_contrato, uni_id, car_id                                |
| ESPECIALIDAD         | esp_id, nombre                                                                                                           |
| ESPECIALIDAD_MEDICO  | med_run, esp_id, fec_ini_espec                                                                                           |
| ATENCION             | ate_id, fecha_atencion, hr_atencion, costo, med_run, esp_id, pac_run                                                     |
| PAGO_ATENCION        | ate_id, fecha_venc_pago, fecha_pago, monto_atencion, monto_a_cancelar, obs_pago                                          |
| PORC_DESCTO_3RA_EDAD | anno_ini, anno_ter, porcentaje_descto (tramos desde 65 hasta 120)                                                        |
| TRAMO_ASIG_ATMED     | tramo_inf_atm, tramo_sup_atm, porc_asig (tramos desde 1 hasta 30 atenciones)                                             |
| PAGO_MOROSO          | pac_run, pac_dv_run, pac_nombre, ate_id, fecha_venc_pago, fecha_pago, dias_morosidad, especialidad_atencion, monto_multa |

**Antes de empezar la actividad:** verifica la carga con `SELECT COUNT(*) FROM ATENCION;` (466 filas) y `SELECT COUNT(*) FROM PACIENTE;` (100 filas). Y no olvides `SET SERVEROUTPUT ON.`